Úvod
- Poznámky k interpretácii relačných diagramov
- Poznámky k interpretácii UML diagramov tried

1. Úvod do JPA - práca s jednou tabuľkou
- Príklad - jednoduchá JDBC aplikácia
- Terminológia JPA
  - Persistence Unit
  - Entity class
  - Entity manager
- Príklad - Prvá JPA aplikácia
  - Čítanie údajov
  - Modifikácia dát
  - Vyhľadávanie podľa kľúča
  - Vytvorenie nového záznamu
- Generovanie tabuľky z entitnej triedy
- Cvičenia

Úvod

Cieľom prednášok je oboznámiť s problematikou objektovo relačného mapovania, štandardom JPA a technológiami poskytujúcimi podporu pre vývoj databázových aplikácií v programovacom jazyku java.

Problematika ORM je obecná - týka sa rovnako všetkých objektovo-orientovaných jazykov. Technológie pre ORM založené na rovnakých princípoch ako JPA možno nájsť v infraštruktúre (ekosystéme) všetkých objektovo-orientovaných jazykov.

Výklad spočíva na ilustračných projektov, je doplnený aj o implementačné úlohy určené pre ďalšie samoštúdium. Pri špecifikácii implementačných úloh budeme využívať relačné diagramy a UML diagramy tried.

Na buildovanie projektov budeme používať nástroj maven a na prednáškach tiež integrované vývojové prostredie Apache Netbeans 23. Poskytuje nielen dobrú podporu pre maven-projekty ale aj klienta pre prácu s databázou a mnoho nástrojov (wizards) pre generovanie entitných tried, vytvorenie JPA konfigurácie. Obdobné nástroje však ponúkajú aj ostatné IDE pre javu (Intellij, Eclipse,...). Maven projekty je možné bez problémov medzi nimi prenášať.

Ako databázu je možné použiť ktorúkoľvek relačnú databázu (poskytujúcu JDBC zavádzač). V ukážkových projektoch budeme prevažne používať PostgreSQL, ktorú považujeme za referenčnú databázu, prípadne Derby DB, ktorá je súčasťou prostredia NetBeans. Pre objektovo-relačné mapovanie budeme používať komponent EclipseLink (verzia 3, príp. vyššia), čo je referenčná implementácia štandardu JPA.

REST služby budeme vyvíjať ako webové aplikácie na platforme Jakarta EE 11 a nasadzovať na aplikačný server Eclipse Glassfish (verzia 7, príp. vyššia).

Upozornenie: Prenesenie ilustračných príkladov na iný aplikačný server, resp. inú verziu platformy (napr. JavaEE) si môže požadovať aj zmeny v zdrojovom kóde.

Maven závislosti

Pre prístup k databáze prostredníctvom JPA je potrebné do projektu samostatnej (standalone) aplikácie pridať nasledujúce závislosti:

```
<!--JPA a EclipseLink-->
   <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.2.0</version>
   </dependency>
   <dependency>
      <groupId>org.eclipse.persistence</groupId>
      <artifactId>org.eclipse.persistence.jpa</artifactId>
      <version>4.0.4</version>
   </dependency>

<!--databazovy driver pre postgres-->
   <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.3</version>
   </dependency>
```

Pre implementáciu REST služby je potrebné do projektu webovej aplikácie vložiť nasledujúce závislosti:

```
<!-- JakartaEE -->
   <dependency>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>11.0.0-M1</version>
      <scope>provided</scope>
   </dependency>
<!-- JAXB -->
   <dependency>
      <groupId>jakarta.xml.bind</groupId>
      <artifactId>jakarta.xml.bind-api</artifactId>
      <version>3.0.1</version>
   </dependency>
```

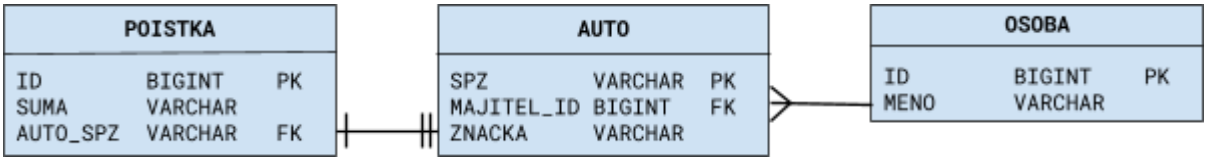
Závislosti pre implementáciu klienta REST služby:

```
    <dependency>
        <groupId>org.glassfish.jersey.core</groupId>
        <artifactId>jersey-client</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>jakarta.xml.bind</groupId>
        <artifactId>jakarta.xml.bind-api</artifactId>
        <version>3.0.1</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-moxy</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-jaxb</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-json-binding</artifactId>
```

```
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.inject</groupId>
        <artifactId>jersey-hk2</artifactId>
        <version>3.1.9</version>
    </dependency>
```

Poznámky k interpretácii relačných diagramov

Pre popis štruktúry databázy budeme používať diagramy s tzv. “crows’ foot” notáciou, ako ilustruje nasledujúca ukážka:



PK označuje primárny kľúč
FK označuje cudzí kľúč

Zakončenie asociácie na strane cudzieho kľúča určuje násobnosť

- '*Kuracie paprčky*' označujú **viacnásobnú asociáciu**: Jedna osoba môže mať viac áut.
- Symbol **|** označuje **jednoduchú asociáciu**: Auto môže mať len jedno (povinné) poistenie (t.j. FK stĺpec musí byť unique)

Symbol **||** na opačnej strane asociácie hovorí, že cudzí kľúč je **povinný** (notNULL). T.j. Poistka musí mať zadanú SPZ auta, ale Auto nemusí mať zadaného majiteľa.

Poznámka: V ER-diagramoch sa zvykne tiež používať symbol **o** pre voliteľné údaje. My budeme namiesto vyznačovania nepovinných FK stĺpcov, označovať povinné FK stĺpce symbolom **||**.

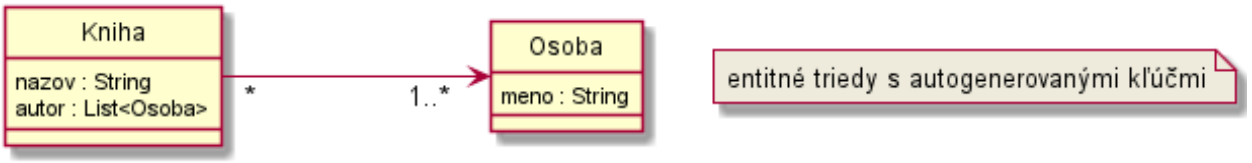
Autogenerované kľúče nie sú v diagrame nijako zvýraznené. Ak má byť kľúč tabuľky autogenerovaný, malo by to byť uvedené explicitne v špecifikácii resp. zadaní.

Poznámky k interpretácii UML diagramov tried

Pozri tiež**:** https://developer.ibm.com/articles/the-class-diagram/

Kľúčové atribúty entitných tried by mali byť uvedené v popise diagramu alebo zobrazené v triede ako prvé a oddelené od ostatných prerušovanou čiarou.

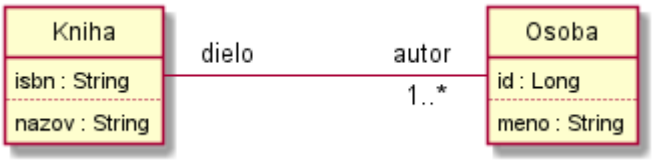
Ak je zo špecifikácie zrejmé, že entitná trieda má autogenerované kľúče, nemusí byť kľúčový atribút v UML diagrame zobrazený.



Referencie na asociovaný/é objekty sa nezvyknú uvádzať medzi dátovými členmi triedy. Názov referencie udáva popis koncového bodu asociácie.

Kardinalitu * resp. `0..*` nemusíme vyznačiť (je to default)

Pri obojsmerných asociáciách (sú bez šípky) by mali mať popis oba konce asociácie.
Potrebujeme totiž názvy referencií na oboch stranách.



Rozdiely medzi ER a OO modelovaním - ER a UML class diagramami

- identita entitných objektov
- atomickosť dát
- násobnosť asociácií (N:N)
- navigovatelnosť (direction) asociácií
- rôzne typy asociácií (kompozícia…)
- dedičnosť

Výklad je rozdelený do šiestich tém, ktoré (približne odpovedajú uvedeným rozdielom a) postupne vysvetľujú hlavné/základné princípy a techniky riešenia problémov vyplývajúcich z rozdielov medzi ER a OO paradigmou.

1. Úvod do JPA - práca s jednou tabuľkou

Prácu s jednou databázovou tabuľkou si najprv ilustrujeme na aplikácii, ktorá číta dáta z databázy pomocou nízkoúrovňového JDBC API.
Následne si aplikáciu reimplementujeme pomocou JPA. Ukážeme pri tom konfiguráciu pripojenia, základnú štruktúru entitnej triedy (trieda mapovaná na databázu), vytvorenie entity managera ilustrujeme jeho použitie.
Nakoniec ukážeme ako je možné pomocou JPA vygenerovať databázové tabuľky k existujúcim entitným triedam.
Na projektoch demonštrujeme aj použitie rôznych IDE a databáz.

Na vytvorenie databázy tabuliek v databáze a naplnenie údajmi pre testovanie aplikácií môžeme použiť akéhokoľvek databázového klienta.
`Netbeans` má takého klienta integrovaného v záložke `Services/Databases`.

Príklad - jednoduchá JDBC aplikácia

Aplikácia bude čítať údaje z tabuľky, ktorú vopred vytvoríme a naplníme údajmi pomocou databázového klienta.

KNIHA		
NAZOV	VARCHAR	PK
CENA	DOUBLE	

Pre prácu s databázou je potrebné najprv získať db-driver a vytvoriť konekciu na databázu

```
Class.forName("org.postgresql.Driver");
Connection con = DriverManager.getConnection(
      "jdbc:postgresql://localhost:5432/edudb", "edu", "edu4edu");
```

Následne môžeme volať SQL-dotazy

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM kniha");
```

Dotaz vracia generickú dátovú štruktúru `ResultSet`, ktorým môžeme postupne prechádzať a pristupovať k načítaným záznamom a položkám.

```
rs.next();
System.out.println(rs.getString("nazov");
```

Ak chceme s knihami pracovať ako a objektami môžeme vytvoriť jednoduchú dátovú triedu (pojo)

```
public class Kniha {
    String nazov;
    double cena;
}
```

a záznamy z generickej dátovej štruktúry manuálne preklopiť do zoznamu objektov tejto triedy

```
        List<Kniha> zoznam = new ArrayList<>();
        while (rs.next()) {
            Kniha k = new Kniha();
            k.nazov = rs.getString(1);
            k.cena = rs.getInt(2);
            zoznam.add(k);
        }

        for (Kniha k : zoznam) {
            System.out.println(k.nazov + " : " + k.cena);
        }
```

Pozri projekt [p1jdbc]

Terminológia JPA

Persistence Unit

Parametre nevyhnutné pre pripojenie a prácu s databázou sú v JPA reprezentované komponentou nazývanou **`Persistence unit.`**
Jej konfigurácia sa nachádza v súbore `persistence.xml,` ktorý je súčasťou projektu.

Entity class

Dátové triedy, ktorých inštancie reprezentujú entity (záznamy) v relačnej databázy nazývame **entitné triedy**.
Minimálne požiadavky, ktoré JPA kladie na entitnú triedu sú

- spĺňa konvencie java bean
    - dátové členy sú java bean property (gettre a settre)
    - konštruktor bez argumentov (môže byť aj default)
- anotácia triedy @Entity
- anotácia @Id pre kľúčovú property

Terminologická poznámka: Inštancie entitnej triedy budeme nazývať aj skrátene **inštancie entity** alebo **entitné objekty**.

Entity manager

Entity manager je komponenta, ktorá zabezpečuje pripojenie na databázu a poskytuje funkcie pre prácu s ňou - synchronizuje stav entitných objektov s entitami (záznamami) v databáze.

Objekt, ktorý je entity managerom prepojený (namapovaný) na entitu v databáze, sa nazýva **manažovaný objekt**.

Kolekcia všetkých objektov manažovaných entity managerom sa nazýva **Persistence context**.
Dôležité: V jednom okamihu môže mať persistence context pre každú databázovú entitu naviac jednu jej inštanciu.

Príklad - Prvá JPA aplikácia

Reimplemenácia predchádzajúceho príkladu pomocou JPA.

Vytvorenie entitnej triedy
Minimálne požiadavky, ktoré JPA kladie na entitnú triedu, sú:

- spĺňa konvencie java bean
    - dátové členy sú java bean property, teda sú verejne prístupné len prostredníctvom getter a setter metód.
    - konštruktor bez argumentov (môže byť aj default)
    - anotácia triedy @Entity
    - anotácia @Id pre dátový člen, ktorý odpovedá kľúčovému stĺpcu.

Dátovú triedu Kniha z predchádzajúceho príkladu možno teda upraviť na entitnú pridaním uvedených anotácií a prístupových metód:

```
@Entity
public class Kniha {
    @Id
    private String nazov;
    private double cena;

    // public access methods
```

```
    public String getNazov() { return nazov; }
    public void setNazov(String nazov) { this.nazov = nazov; }
    public double getCena()  {return cena; }
    public void setCena(double cena) { this.cena = cena; }
}
```

Žiadne ďalšie úpravy nie sú potrebné. Všimnite si, že názov triedy je zhodný s názvom tabuľky, takisto názvy dátových členov sa zhodujú s názvami stĺpcov. Vďaka tomu dokáže JPA automaticky priradiť (namapovať) triedu tabuľke a dátové členy jej stĺpcom.

Pripojenie k databáze

Údaje pre pripojenie k databáze zadáme do elementu **`persistence-unit`**, ktorý sa nachádza v súbore **`persistence.xml`**. V podelemente class je tiež potrebné uviesť názvy entitných tried mapovaných na túto databázu.

```
<persistence-unit name="vsaPU" transaction-type="RESOURCE_LOCAL">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <class>vsa.Kniha</class>
  <properties>
    <property name="jakarta.persistence.jdbc.url"
              value="jdbc:postgresql://localhost:5432/edudb"/>
    <property name="jakarta.persistence.jdbc.user" value="edu"/>
    <property name="jakarta.persistence.jdbc.driver"
              value="org.postgresql.Driver"/>
    <property name="jakarta.persistence.jdbc.password" value="edu4edu"/>
  </properties>
</persistence-unit>
```

Poznámka. V prostredí netbeans môžeme súbor `persistence.xml` vytvoriť wizardom `New/Persistence Unit` v kontextovom menu projektu.

Pripojenie a prístup k databáze zabezpečuje manažér entít. Vytvoríme ho v aplikácii pomocou faktory metódy, ktorej ako argument zadáme meno elementu `persistence-unit`. V našom príklade: `"vsaPU"`.

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");
EntityManager em = emf.createEntityManager();
```

Nasledujúce funkcie ilustrujú prácu s databázou pomocou metód EntityManagera.

Čítanie údajov - reimplementácia JDBC aplikácie pomocou JPA

Manažér entít poskytuje metódu `createNativeQuery`, ktorá umožňuje vytvoriť SQL dotaz na čítanie údajov z databázy.

```
Query q = em.createNativeQuery("select * from kniha", Kniha.class);
List<Kniha> knihy = q.getResultList();

for (Kniha k : knihy) {
  System.out.println("" + k);
}
```

Implementáciu príkladu nájdete v projekte **p1jpa**.
Čítanie údajov je v projekte implementované funkciou `citajKnihy`. Projekt je doplnený aj o funkcie `zmenCeny`, `najdiKnihu`, `vytvorKnihu` ilustrujúce prácu s transakciami a metódy manažéra `find` a `persist`.

Modifikácia dát

Ilustruje `getTransaction, begin, commit`

```
Query q = em.createNativeQuery("select * from kniha", Kniha.class);
List<Kniha> knihy = q.getResultList();
em.getTransaction().begin();
for (Kniha k : knihy) {
  k.setCena(k.getCena() * 0.8);
}
em.getTransaction().commit();
```

Vyhľadávanie podľa kľúča

```
Kniha k = em.find(Kniha.class, "Pipi");
System.out.println("" + k);
```

Vytvorenie nového záznamu

```
Kniha k = new Kniha();
k.setNazov("Brave new world");
em.getTransaction().begin();
em.persist(k);
em.getTransaction().commit();
```

Generovanie tabuľky z entitnej triedy

Príklad ilustruje **zmenu databázy** a tiež **generovanie tabuľky** z entitnej triedy pomocou JPA. Použiť môžeme priamo projekt **p1jpa** alebo jeho kópiu, ktorú upravíme:

- do `pom.xml` pridáme závislosť pre driver databázy derbyDB.

```
<dependency>
   <groupId>org.apache.derby</groupId>
   <artifactId>derbyclient</artifactId>
   <version>10.15.2.0</version>
</dependency>
```

- úpravíme `persistence-unit` pre databázu derbyDB.

```
<persistence-unit name="vsaPU" transaction-type="RESOURCE_LOCAL">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <properties>
    <property name="jakarta.persistence.jdbc.url"
      value="jdbc:derby://localhost:1527/sample"/>
    <property name="jakarta.persistence.jdbc.user" value="app"/>
    <property name="jakarta.persistence.jdbc.driver"
      value="org.apache.derby.client.ClientAutoloadedDriver"/>
    <property name="jakarta.persistence.jdbc.password" value="app"/>
    <property name="jakarta.persistence.schema-generation.database.action"
      value="create"/>
  </properties>
</persistence-unit>
```

V tejto databáze zatiaľ tabuľka KNIHA neexistuje. Nastavíme preto v elemente `schema-generation.database.action` hodnotu `create` (príp. `drop-and-create`).
Po spustení funkcie `vytvorKnihu()` si môžeme overiť, že tabuľka bola v databáze vytvorená.

Hodnoty **schema-generation.database.action**

- `none` žiadna akcia
- `create` pred spustením aplikácie vytvorí JPA tabuľku pre entitnú triedu, ak neexistovala.
- `drop-and-create` pred spustením najprv odstráni existujúcu a znovu vytvorí prázdnu tabuľku. Vhodné je to pre testovanie a ladenie:

- pri opakovanom spustení `vytvorKnihu()` nenastane unique-key-constraint violation.
- pri zmene štruktúry entitnej triedy - modifikuje aj štruktúru tabuľky.

Cvičenia

V nasledujúcich úlohách vytvárame rôzne modifikácie projektu **p1jpa** z prednášky.

CV1. Otvorte a pracujte s projektom p1jpa v inom IDE *(musí podporovať maven projekty)*

CV2. Použite pre projekt s inú SQL-databázou, napr. derbyDB *(je integrovaná v prostredí Apache Netbeans)* alebo mySQL.

CV3. Entitná trieda a tabuľka s rôznou štruktúrou
Cieľom cvičenia je zistiť čo sa stane, ak entitná trieda a tabuľka nemajú rovnakú štruktúru dát.
V projekte p1jpa urobte tieto úpravy:

- Do triedy `Kniha` pridajte novú property `String autor`. Do tabuľky KNIHA však stĺpec pre autora nedávajte.
- Do tabuľky KNIHA pridajte nový stĺpec `ISBN` typu `VARCHAR`. Do triedy `Kniha` však property pre ISBN nedávajte.
- Schema-generation ponechajte `none`
- Použite funkciu `citajKnihy()` na čítanie kníh z databázy. Vysvetlite čo sa stane?
- Použite funkciu `vytvorKnihu()` pre vloženie novej knihy. Vysvetlite čo sa stane?
- V triede `Kniha` dajte dátovému členu `autor` anotáciu **`@Transient`** a opäť spustite funkciu na čítanie kníh z databázy. Čo sa stane teraz?
  Vysvetlenie: *Dátový člen entitnej triedy, ktorý je anotovaný* `@Transient`, *nebude namapovaný na žiadny stĺpec tabuľky.*

CV4. Pregenerovanie databázy po zmene štruktúry entitnej triedy.
V projekte p1jpa urobte tieto úpravy:

- Do triedy `Kniha` **pridajte novú property** `String autor`. *(Tabuľku KNIHA ponechajte bez zmien, má len stĺpce nazov, cena)*
- Nastavte schema-generation strategy na **`drop-and-create`**.
- Použite funkciu na vloženie novej knihy do databázy. Presvedčte sa, že tabuľka KNIHA bola nanovo vytvorená a má aktualizovanú štruktúru.

CV5. Mapovanie dátových členov enumeračného typu
V projekte p1jpa

- vytvorte enumeračný typ popisujúci dostupnosť knihy
  `enum Dostupnost { NASLKADE, NAOBJEDNAVKU, VYPREDANA };`
  a pridajte do triedy `Kniha` atribút `Dostupnost dostupnost;`
- Schema-generation strategy ponechajte `drop-and-create`.
- Upravte a spustite program na vloženie novej knihy, zadajte hodnotu pre dostupnosť . Zistite aký typ dát obsahuje nový stĺpec `DOSTUPNOST`
- Následne aribútu `dostupnost` dajte anotáciu **`@Enumerated(EnumType)`**
  Spustite znovu funkciu na vloženie novej knihy a pozrite, aký typ dát obsahuje nový stĺpec `DOSTUPNOST` teraz.
- Vyskúšajte tiež, čo bude obsahovať stĺpec, ak v novej knihe hodnotu pre `dostupnost` nezadáte.

CV6. Rozdiely v mapovaní dátových členov typu double a triedy Double
V projekte p1jpa urobte tieto úpravy:

- V tabuľke `KNIHA` vytvorte pomocou databázového klienta novú knihu, ktorá nemá zadanú cenu.
  Poznámka: Ak stĺpec `CENA` nemá povolené hodnoty `NULL`, povoľte ich *(vo vlastnostiach stĺpca)*.
- Stratégiu pre generovanie schémy nastavte na `none`.
- V aplikácii načítajte novovytvorenú knihu a presvedčte sa, že dátový člen `cena` obsahuje hodnotu `0.0`.
  Vysvetlenie: Dátový člen cena je totiž hodnota typu `double` a nie objekt triedy `Double`.
- V triede Kniha zmeňte typ dátového člena cena na `Double` a načítajte vytvorenú knihu znovu. Presvedčte sa, že teraz je cena novej knihy `null`.

CV7. Generovanie stĺpcov s povolenými hodnotami NULL

- Ak tabuľku KNIHA vytvoríme z entitnej triedy `Kniha` (pomocou schema-generation `create` alebo `drop-and-create`) stĺpec CENA má nulové hodnoty povolené. Presvedčte sa o tom.
- V triede Kniha pridajte atribútu cena anotáciu

```
@Column(nullable=false)
private double cena;
```

- Stratégiu generovania databázy ponechajte drop-and-create.
  Spustite aplikáciu aby sa tabuľka nanovo vygenerovala a presvedčte sa, že teraz stĺpec CENA nemá povolené hodnoty NULL.
  Poznámka: Môžete si tiež overiť, že stĺpec bude notNULL aj keby bol atribút cena typu Double (teda umožnoval aj hodnotu null).
- Zmeňte typ atribútu `cena` na `Double`.

```
@Column(nullable=false)
private Double cena;
```

- Vytvorte v programe nový objekt triedy `Kniha`, ktorý nemá zadanú `cenu` a skúste ho vložiť do databázy. Čo sa stane?

CV8. unique stĺpce

- Pridajte do triedy `Kniha` atribút `String ISBN`, s anotáciou:

```
@Column(unique=false)
private String isbn;
```

- Schema-generation strategy ponechajte `drop-and-create`.
  Spustite aplikáciu a pozrite sa, akým spôsobom sa v štruktúre databázy prejavila unikátnosť stĺpca `ISBN` (mal by byť pre neho vytvorený index s vlastnosťou `unique`).
- Upravte program, aby ste si overili, že do databázy nemozno vložiť dve knihy s rovnakým isbn.

CV9. zhrnutie

Pridajte do triedy `Kniha` atribúty `ISBN, autor, dostupnost`.
Upravte anotácie stĺpcov , tak aby po pregenerovaní tabuľky KNIHA, bol primárny kľúč stĺpec ISBN, stĺpec NAZOV bol unikátny a nepovolil NULL hodnoty a ostatné stĺpce NULL povolili.
Upravte a otestujete všetky funkcie projektu.